

Leonardo Domingues de Lima e Carvalho Filho [568220]
Jullyemerson Bento Leonizio [541778]
William Sousa Lavor [556873]

Trabalho FreeBSD

Sistemas Operacionais

Professor Caio Cesar

1 Gerenciamento de Processos no FreeBSD

1.1 Conceito de processo no sistema analisado:

No FreeBSD, um processo é uma instância de um programa em execução. Cada processo possui um identificador único chamado PID, além de informações como usuário proprietário, prioridade, estado e recursos utilizados. Internamente, o kernel gerencia os processos por meio da estrutura struct proc, que armazena todos os dados necessários para o controle e escalonamento. Todo processo, exceto o init (PID 1), possui um processo pai.

Referência:

<https://docs.freebsd.org/en/books/handbook/basics/#processes>

1.2 Criação e finalização de processos:

A criação de processos no FreeBSD ocorre principalmente através da chamada de sistema fork(), que cria um processo filho como cópia do processo pai. Após o fork(), o processo filho normalmente utiliza exec() para carregar um novo programa na memória. A finalização ocorre quando o processo chama exit(), liberando seus recursos. Após finalizar, o processo entra no estado zumbi até que o processo pai chame wait() ou waitpid().

Exemplo em C:

```
pid_t pid = fork();
if (pid == 0) {
    execl("/bin/ls", "ls", NULL);
    _exit(1);
} else {
    wait(NULL);
}
```

Referências:

<https://man.freebsd.org/cgi/man.cgi?fork>

<https://man.freebsd.org/cgi/man.cgi?execve>

<https://man.freebsd.org/cgi/man.cgi?exit>

1.3 Estados de processo:

Os processos no FreeBSD podem assumir diversos estados, observáveis por ferramentas como ps e top. Os principais são:

Alguns exemplos de estados:

RUN (Running): processo em execução ou pronto para executar.

SLEEP (Sleeping): aguardando algum evento ou recurso.

STOP (Stopped): interrompido por sinal.

ZOMB (Zombie): processo finalizado aguardando coleta pelo pai.

Esses estados permitem ao kernel controlar o uso da CPU e recursos do sistema.

Referência:

<https://man.freebsd.org/cgi/man.cgi?ps>

1.4 Relação entre processos e aplicações:

No FreeBSD, cada aplicação em execução corresponde a um ou mais processos. Ao executar um comando no terminal, o shell cria um processo para a aplicação. O sistema suporta multitarefa preemptiva, permitindo que vários processos sejam executados simultaneamente. Aplicações podem rodar em primeiro plano ou em segundo plano (background), e serviços do sistema geralmente executam como processos daemon.

Referência:

<https://docs.freebsd.org/en/books/handbook/basics/#process-control>

2 Threads

2.1 Suporte a threads no sistema operacional:

O FreeBSD oferece suporte completo a threads, integradas diretamente ao kernel. O modelo adotado é conhecido como **1:1**, no qual cada thread em nível de usuário corresponde a uma thread em nível de kernel. Isso permite que o escalonador do sistema trate threads individualmente, distribuindo-as entre múltiplos núcleos de CPU e garantindo melhor desempenho em sistemas multiprocessados.

O suporte a threads no FreeBSD é amplamente utilizado por meio da biblioteca POSIX Threads (pthreads), que fornece uma interface padronizada para criação, sincronização e finalização de threads em aplicações. Internamente, o kernel gerencia threads utilizando a estrutura struct thread, associada a um processo.

Referências:

<https://man.freebsd.org/cgi/man.cgi?query=Thread&sektion=3&manpath=FreeBSD+14.1-RELEASE+and+Ports>

2.2 Diferença entre processos e threads no sistema:

No FreeBSD, processos e threads diferem principalmente no nível de isolamento e no compartilhamento de recursos. Um processo possui seu próprio espaço de endereçamento, descritores de arquivos e contexto de execução isolado. Já uma thread é uma unidade de execução mais leve, que compartilha com outras threads do mesmo processo recursos como memória, arquivos abertos e variáveis globais..

Referências:

https://man.freebsd.org/cgi/man.cgi?pthread_create

2.3 Threads em nível de usuário e/ou kernel (quando aplicável):

O FreeBSD utiliza exclusivamente threads em nível de kernel, seguindo o modelo 1:1. Em versões antigas, existiam implementação de threads em nível de usuário, mas elas foram descontinuadas devido a limitações de desempenho e escalabilidade.

Referências:

<https://docs.freebsd.org/en/articles/linux-emulation/#threads>

3 Escalonamento de CPU

3.1 Tipo de escalonamento (preemptivo ou não preemptivo):

O FreeBSD utiliza escalonamento preemptivo, no qual o kernel pode interromper a execução de um processo ou thread para permitir que outro seja executado. Esse modelo garante melhor resposta do sistema, especialmente em ambientes multitarefa e multiprocessados. A preempção permite que processos de maior prioridade obtenham acesso imediato à CPU, evitando que tarefas longas monopolizem o processador. Esse comportamento é fundamental para sistemas interativos e servidores, onde o tempo de resposta é crítico.

Referências:

<https://docs.freebsd.org/en/books/handbook/config/#kernelconfig-scheduler>

3.2 Algoritmo(s) de escalonamento utilizado(s):

O principal algoritmo de escalonamento utilizado no FreeBSD é o ULE Scheduler, que substituiu o antigo escalonador 4BSD. O ULE foi projetado para sistemas multiprocessados e multicore, oferecendo melhor balanceamento de carga entre CPUs, ele mantém filas de execução por CPU, reduzindo a contenção e melhorando a escalabilidade. Além disso, o FreeBSD também oferece suporte a classes de escalonamento especiais, como a classe Real-Time (RT), destinada a aplicações que exigem prazos rígidos de execução.

Referências:

<https://docs.freebsd.org/en/books/handbook/config/#scheduling>

https://man.freebsd.org/cgi/man.cgi?sched_ule

3.3 Critérios adotados (prioridade, justiça, tempo real, desempenho):

Prioridade: Cada processo ou thread possui uma prioridade dinâmica, que pode variar conforme o comportamento de uso da CPU. Processos interativos tendem a receber prioridade maior para melhorar a responsividade.

Justiça: ULE(escalonador) busca distribuir o tempo de CPU de forma equilibrada entre os processos, evitando starvation (inanição) de tarefas com menor prioridade.

Tempo real: Processos configurados com prioridades de tempo real podem “preemptar” processos comuns, sendo usados em aplicações críticas que exigem execução imediata.

Desempenho: ULE considera afinidade de CPU e uso de cache, reduzindo migrações desnecessárias e melhorando a eficiência geral do sistema.

Referências:

<https://docs.freebsd.org/en/books/handbook/basics/#scheduling>

<https://man.freebsd.org/cgi/man.cgi?rtprio>

4 Gerenciamento de Memória

4.1 Existência de memória virtual:

O FreeBSD implementa um sistema completo de vm(memória virtual), permitindo que cada processo utilize um espaço de endereçamento virtual independente da memória física disponível. O subsistema de memória virtual do FreeBSD é responsável pelo mapeamento entre endereços virtuais e físicos, pelo controle de permissões de acesso e pela otimização do uso da memória por meio de técnicas como compartilhamento de páginas e carregamento sob demanda.

Referências:

<https://docs.freebsd.org/en/books/arch-handbook/vm/>

4.2 Uso de paginação e/ou segmentação:

O FreeBSD utiliza paginação como principal técnica de gerenciamento de memória. A memória virtual é dividida em páginas de tamanho fixo, que são mapeadas para quadros de memória física conforme necessário. O carregamento das páginas ocorre sob demanda (demand paging), reduzindo o uso inicial de memória e melhorando o desempenho geral do sistema.

A segmentação, no sentido clássico, não é utilizada como mecanismo principal. No entanto, conceitos de segmentação ainda podem existir em nível de arquitetura de hardware (como em processadores x86), sendo abstraídos pelo kernel. Para o sistema operacional, o modelo predominante é totalmente baseado em paginação.

Referências:

<https://docs.freebsd.org/en/books/arch-handbook/vm/>

<https://man.freebsd.org/cgi/man.cgi?vm>

4.3 Mecanismos de swap ou equivalentes:

FreeBSD utiliza-se de swap para expandir a memória disponível, transferindo páginas pouco utilizadas da memória física para dispositivos de armazenamento secundário. FreeBSD permite que o swap possa ser configurado tanto em partições quanto em arquivos, oferecendo flexibilidade ao administrador do sistema.

Referências:

<https://man.freebsd.org/cgi/man.cgi?swapon>

4.4 Proteção e isolamento de memória entre processos:

O sistema operacional garante proteção e isolamento de memória por meio do uso de espaços de endereçamento virtual separados para cada processo. Um processo não pode acessar diretamente a memória de outro, a menos que mecanismos explícitos de compartilhamento sejam utilizados, como memória compartilhada (shared memory). O kernel, em conjunto com o hardware de gerenciamento de memória (MMU), aplica permissões de acesso às páginas, controlando leitura, escrita e execução. Qualquer tentativa de acesso inválido resulta em uma falha de segmentação (segmentation fault), protegendo a integridade do sistema.

Referências:

<https://docs.freebsd.org/en/books/handbook/security/#memory-protection>

5 Gerenciamento de Armazenamento

5.1 Sistemas de arquivos utilizados:

O FreeBSD suporta diversos sistemas de arquivos, sendo os principais o UFS (Unix File System) e o ZFS (Zettabyte File System). O UFS é o sistema de arquivos tradicional do FreeBSD, amplamente utilizado por sua simplicidade e estabilidade. Já o ZFS é um sistema moderno, integrado a um gerenciador de volumes, oferecendo alta confiabilidade, escalabilidade e recursos avançados de integridade de dados.

Além desses, o FreeBSD também oferece suporte a outros sistemas de arquivos, como FAT, NTFS (somente leitura ou com suporte limitado), NFS para acesso remoto e tmpfs para sistemas de arquivos em memória.

Referências:

<https://docs.freebsd.org/en/books/handbook/filesystems/>

<https://man.freebsd.org/cgi/man.cgi?zfs>

5.2 Organização e alocação de arquivos:

No FreeBSD, a organização de arquivos segue o modelo hierárquico de diretórios típico de sistemas Unix. Os arquivos são representados internamente por inodes, que armazenam metadados como permissões, proprietário, tamanho e ponteiros para os blocos de dados no disco.

No UFS, a alocação de blocos é feita de forma a manter dados relacionados fisicamente próximos, reduzindo a fragmentação e melhorando o desempenho. Já no ZFS, a alocação é baseada em copy-on-write, no qual os dados nunca são sobrescritos diretamente, mas gravados em novos blocos, aumentando a confiabilidade e evitando a corrupção de dados.

Referências:

<https://docs.freebsd.org/en/books/handbook/filesystems/#ufs>

<https://docs.freebsd.org/en/books/handbook/zfs/>

5.3 Suporte a journaling ou mecanismos de recuperação:

O FreeBSD oferece mecanismos de recuperação para garantir a consistência do sistema de arquivos após falhas.

No UFS, esse papel é desempenhado pelo Soft Updates, que mantém a consistência das estruturas do sistema de arquivos sem a necessidade de journaling tradicional, reduzindo o tempo de verificação com fsck após desligamentos inesperados.

O ZFS, por sua vez, não utiliza journaling convencional, pois implementa um modelo transacional baseado em copy-on-write, garantindo que o sistema de arquivos esteja sempre em um estado consistente. Além disso, o ZFS possui verificação automática de integridade com checksums e capacidade de auto recuperação quando usado com redundância.

Referências:

<https://docs.freebsd.org/en/books/handbook/filesystems/#soft-updates>

<https://docs.freebsd.org/en/books/handbook/zfs/#zfs-reliability>

6 Chamadas de Sistema (System Calls)

6.1 Conceito de chamadas de sistema no SO analisado:

No FreeBSD, chamadas de sistema (system calls) são o principal mecanismo pelo qual programas em espaço de usuário solicitam serviços ao kernel. Como aplicações não têm acesso direto ao hardware nem às estruturas internas do sistema operacional, as system calls funcionam como uma interface controlada e segura entre o modo usuário e o modo kernel. Cada chamada de sistema executa uma função específica do kernel, como criação de processos, acesso a arquivos, comunicação entre processos ou gerenciamento de memória. Internamente, o FreeBSD mantém uma tabela de chamadas de sistema, que associa números de system calls às rotinas correspondentes no kernel.

Referências:

<https://docs.freebsd.org/en/books/handbook/basics/#system-calls>

6.2 Exemplos de operações realizadas via system calls:

Diversas operações fundamentais do sistema são realizadas por meio de chamadas de sistema no FreeBSD.

Alguns exemplos de operações:

Gerenciamento de processos: `fork()`, `execve()`, `wait()`, `exit()`

Gerenciamento de arquivos: `open()`, `read()`, `write()`, `close()`

Gerenciamento de memória: `mmap()`, `munmap()`, `brk()`

Comunicação entre processos: `pipe()`, `shm_open()`, `socket()`

Controle de dispositivos: `ioctl()`

Essas chamadas são geralmente acessadas por aplicações através de bibliotecas padrão, como a `libc`, que fornece funções de alto nível para facilitar o uso das system calls.

Referências:

<https://man.freebsd.org/cgi/man.cgi?syscall>

<https://man.freebsd.org/cgi/man.cgi?open>

<https://man.freebsd.org/cgi/man.cgi?fork>

6.3 Interface utilizada (POSIX, APIs próprias, etc.):

FreeBSD adota a interface POSIX, garantindo compatibilidade com aplicações desenvolvidas para outros sistemas do tipo Unix, como Linux e macOS. A maioria das chamadas de sistema segue rigorosamente os padrões POSIX, o que facilita a portabilidade de software. Além do POSIX, o FreeBSD também oferece extensões e APIs próprias, que permitem acesso a funcionalidades específicas do sistema, como recursos avançados de rede, gerenciamento de eventos com `kqueue` e controle detalhado de processos e dispositivos. Essas extensões complementam o padrão POSIX sem comprometer a compatibilidade.

Referências:

<https://man.freebsd.org/cgi/man.cgi?kqueue>

7 Relação com a Teoria

7.1 Comparação entre o que foi estudado em sala e o que é implementado no sistema:

Os conceitos estudados em sala de aula sobre sistemas operacionais como processos, threads, escalonamento de CPU, memória virtual, chamadas de sistema e sistemas de arquivos são diretamente aplicáveis ao FreeBSD. O sistema implementa de forma prática os modelos teóricos clássicos apresentados na literatura, como a separação entre modo usuário e modo kernel, o uso de memória virtual com paginação e o escalonamento preemptivo.

Os conceitos de processos e threads são claramente percebidos no FreeBSD, ambos são materializados por estruturas internas do kernel e por APIs padronizadas, como POSIX. Da mesma forma, algoritmos de escalonamento e políticas de prioridade discutidos em sala são implementados por meio do escalonador ULE e das classes de escalonamento do sistema.

Referências:

<https://www.freebsd.org/doc/en/books/design-44bsd/>

<https://docs.freebsd.org/en/books/handbook/>

7.2 Vantagens e limitações das escolhas adotadas pelo sistema operacional:

As escolhas de projeto adotadas pelo FreeBSD apresentam diversas vantagens. O uso de escalonamento preemptivo e threads em nível de kernel garante boa responsividade e aproveitamento de sistemas multiprocessados. O subsistema de memória virtual oferece isolamento, proteção e eficiência, enquanto o suporte a sistemas de arquivos avançados, como o ZFS, proporciona alta confiabilidade e integridade dos dados. Além disso, a forte adesão ao padrão POSIX facilita a portabilidade de aplicações. Por outro lado, algumas limitações podem ser observadas. O uso de threads em nível de kernel pode gerar maior sobrecarga em comparação a modelos puramente em nível de usuário em aplicações muito simples. O ZFS, apesar de suas vantagens, exige maior consumo de memória. Além disso, por priorizar estabilidade e consistência, o FreeBSD pode adotar inovações de forma mais conservadora quando comparado a outros sistemas operacionais.

Referências:

<https://docs.freebsd.org/en/books/handbook/introduction/>

<https://www.freebsd.org/doc/en/books/design-44bsd/>